

Malware Analysis Practical - Exam Ready 15-Mark Answers

CDS-306-MJP Practical based on CDS-302-MJ Malware Analysis

Use actual practical values for hashes, detection ratios, IPs, URLs and observations.

Sheet 1

Q1. Analyze the given executable file using VirusTotal. Prepare a report including the detection ratio, file hash (MD5/SHA256), file type, and security vendors' results. [15M]

Answer

Aim: To check whether the given executable is detected as malicious and collect its basic identification details using VirusTotal.

Tools Required: VirusTotal website, web browser, given executable file.

Safe Setup: If the sample is confidential, follow the examiner/lab policy before uploading it to an online service.

Step-by-Step Procedure

1. Open VirusTotal and choose the File analysis option.
2. Upload the executable provided in the practical. If upload is not allowed, calculate its SHA-256 and search the hash instead.
3. Wait for the scan to complete and open the Detection tab.
4. Note the detection ratio, for example: ___ / ___ security vendors detected the file.
5. Open the Details tab and record MD5 and SHA-256 hash values.
6. Record the file type, size, architecture or format shown by VirusTotal.
7. From the Detection tab, note 4-6 important vendor results such as Microsoft, Kaspersky, ESET, BitDefender, Sophos, etc.
8. Check Details / Relations / Behavior only for extra useful information such as names, contacted domains, or dropped files if available.
9. Take screenshots of the main Detection page and Details page for evidence.
10. Write the final conclusion only from the observed results; a high detection ratio strongly suggests malware, while a low ratio needs further analysis.

What to Observe / Record

- Detection ratio
- MD5 hash
- SHA-256 hash
- File type / format
- File size
- Important vendor verdicts
- Any obvious IOCs shown by the report

Report Format

Field	Observed value
File name	_____
Detection ratio	___ / ___
MD5	_____
SHA-256	_____
File type	_____
File size	_____
Vendor 1 result	_____
Vendor 2 result	_____
Vendor 3 result	_____
Final verdict	Malicious / Suspicious / No clear detection

How to generate the report: Create a heading with sample name and date, copy the actual values from VirusTotal into the table, attach 1-2 screenshots, then write a 2-3 line conclusion. Do not write a detection ratio or hash that you did not actually obtain.

Result / Conclusion: The executable was checked using VirusTotal and its detection ratio, hashes, file type, and vendor results were recorded. The final verdict is based on the actual scan results.

Q2. Explain the concept of malware and describe different types of malware with suitable examples. [15M]

Answer

Malware means malicious software. It is software intentionally designed to damage a system, steal data, spy on users, disrupt services, or gain unauthorized access. Malware can enter through infected files, malicious links, software vulnerabilities, removable media, or unsafe downloads.

- Virus: Attaches itself to a legitimate file or program and spreads when the infected file is executed. Example: file-infecting viruses such as Sality.
- Worm: Self-replicates and spreads over networks without needing a host file. Example: Conficker; WannaCry also used worm-like spreading.
- Trojan: Looks like legitimate software but performs malicious actions after installation. Example: Zeus banking Trojan.
- Ransomware: Encrypts files or locks a system and demands payment. Example: WannaCry ransomware.
- Spyware: Secretly collects user information such as browsing activity, credentials, or documents.
- Keylogger: Records keystrokes to steal passwords, messages, or banking details.
- Rootkit: Hides malicious processes/files and helps an attacker maintain privileged access.
- Backdoor: Creates a hidden method for remote access that bypasses normal authentication.
- Bot/Botnet malware: Turns the infected computer into a remotely controlled bot that can be used for DDoS, spam, or other attacks.
- Adware / Potentially unwanted software: Displays unwanted advertisements and may track user activity; not all adware is equally harmful.

Common effects of malware include slow system performance, unknown processes, disabled security tools, file encryption, data theft, network connections to unknown servers, and unauthorized changes to files or registry settings.

Conclusion: Malware is a broad category. Its type is classified mainly by how it spreads, what it does, and how it maintains control over the victim system.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 2

Q1. Analyze the given executable using PEStudio and identify the file information, imported libraries, suspicious indicators, and strings. Prepare a report. [15M]

Answer

Aim: To perform basic static analysis of a Windows executable without running it.

Tools Required: PEStudio and the given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Launch PEStudio and open the given executable.
2. Check the Overview / file information section and note file name, size, hashes, file type, architecture, compile timestamp, and entry point if shown.
3. Open the Libraries section and list imported DLLs such as KERNEL32.dll, USER32.dll, ADVAPI32.dll, WININET.dll, WS2_32.dll, etc.
4. Open Imports and look for suspicious APIs related to process creation, memory allocation, registry modification, downloading, networking, or persistence.
5. Open Indicators and review entries highlighted with warning or suspicious scores. Do not treat one indicator alone as proof of malware.
6. Open Strings and search for URLs, IP addresses, file paths, registry paths, commands, mutex names, suspicious extensions, and readable messages.
7. Check Sections for unusual names, high entropy, very small/large sections, or executable/writable permissions if PEStudio shows them.
8. Take screenshots of Overview, Imports/Indicators, and Strings.
9. Summarize the suspicious findings and write a verdict such as "requires further analysis" if evidence is not conclusive.

What to Observe / Record

- File identity and hashes
- Imported DLLs
- Suspicious Windows APIs
- PEStudio indicators
- Interesting strings
- Section anomalies

Report Format

Category	Finding
File information	_____
Imported DLLs	_____
Suspicious APIs	_____
Indicators	_____
Suspicious strings	_____
Section findings	_____
Verdict	_____

How to generate the report: Start with file information, then list imports, indicators, strings, and section findings. For each suspicious item, write one short reason. Finish with a cautious conclusion based on the total evidence.

Result / Conclusion: Static properties of the executable were examined in PEStudio and the important imports, indicators, and strings were documented.

Q2. Differentiate between Malware, Vulnerability, and Exploit with suitable examples. [15M]

Answer

Malware, vulnerability, and exploit are related security terms, but they are not the same.

Point	Malware	Vulnerability	Exploit
-------	---------	---------------	---------

Meaning	Malicious software designed to cause harm or unauthorized activity.	A weakness or flaw in software, hardware, configuration, or process.	Code or a technique that takes advantage of a vulnerability.
Purpose	To steal, damage, spy, disrupt, or gain control.	It has no purpose by itself; it is a weakness that creates risk.	To trigger the weakness and obtain an unintended effect.
Example	Ransomware, Trojan, spyware.	An unpatched buffer overflow or weak default password.	A crafted input that abuses the buffer overflow to execute code.
Existence	An actual malicious program or component.	May exist even if nobody attacks it.	Usually depends on a specific vulnerability or misconfiguration.
Defense	Antivirus/EDR, allowlisting, backups, monitoring.	Patching, secure configuration, vulnerability management.	Patch the weakness, block exploit techniques, monitor attack attempts.

Simple relationship: Vulnerability = weakness; Exploit = method used to abuse the weakness; Malware = malicious software that may be delivered or executed after successful exploitation. Example: A browser flaw is the vulnerability, exploit code triggers it, and a Trojan may then be installed.

Conclusion: A vulnerability creates the opportunity, an exploit uses that opportunity, and malware is the malicious software that performs harmful actions.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 3

Q1. Perform Strings Analysis on the given executable using the Strings utility and identify suspicious file paths, URLs, registry keys, and commands. [15M]

Answer

Aim: To extract readable text from the executable and identify clues about possible malware behavior.

Tools Required: Sysinternals Strings (or equivalent strings utility), Command Prompt/PowerShell, given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Place the sample in the analysis folder inside the lab system/VM.
2. Open Command Prompt in the folder containing Strings and the sample.
3. Run a command such as: `strings.exe sample.exe > strings.txt`. For Unicode strings, also use the appropriate Unicode option if required by the tool version.
4. Open strings.txt in a text editor.
5. Search for URL keywords such as `http`, `https`, `www`, `.com`, `.net`, `.ru`, etc.
6. Search for Windows file paths such as `C:\Users`, `AppData`, `Temp`, `System32`, `ProgramData`, or unusual executable/script paths.
7. Search for registry paths such as `HKCU`, `HKLM`, `Software\Microsoft\Windows\CurrentVersion\Run`, or service-related paths.
8. Search for command strings such as `cmd.exe`, `powershell`, `rundll32`, `reg`, `schtasks`, `wmic`, `curl`, `certutil`, or script commands.
9. Classify only meaningful strings and ignore random unreadable output.
10. Record the suspicious strings exactly as observed and explain briefly why each one may be relevant.

What to Observe / Record

- URLs/domains
- IP-like strings
- File paths
- Registry keys
- Commands
- User-agent strings
- Error/debug messages
- Possible mutex or service names

Report Format

Type	Extracted string	Why it is important
URL / Domain	_____	Possible network destination
File path	_____	Possible drop/execute location
Registry key	_____	Possible persistence/configuration
Command	_____	Possible executed command

How to generate the report: Attach the strings.txt output or screenshot, then create the classification table. Do not assume every string is malicious; explain whether it is suspicious only in context.

Result / Conclusion: Readable strings were extracted and classified into possible network, file, registry, and command indicators for further malware analysis.

Q2. Explain the malware lifecycle with a suitable diagram. [15M]

Answer

A malware lifecycle describes the common stages through which malicious software reaches a victim, gains execution, maintains access, communicates, and performs its objective. Different malware families may skip or combine some stages.

**Delivery -> Execution/Exploitation -> Installation -> Persistence -> Command & Control -> Malicious Actions
-> Propagation / Cleanup**

- **Delivery:** Malware reaches the target through phishing email, malicious attachment, unsafe download, compromised website, removable media, or another route.
- **Execution / Exploitation:** The victim runs the file or an exploit causes malicious code to execute.
- **Installation:** The malware copies itself, drops files, creates components, or modifies the system.
- **Persistence:** It tries to survive reboot or user logoff using startup entries, services, scheduled tasks, startup folders, etc.
- **Command and Control (C2):** Some malware contacts an external server or peer to receive instructions or send stolen information.
- **Malicious Actions:** It performs its objective, such as credential theft, file encryption, spying, data destruction, or remote control.
- **Propagation:** Worms or some ransomware attempt to spread to other systems using networks, shared drives, credentials, or removable media.
- **Cleanup / Evasion:** Some malware deletes temporary files, changes names, hides artifacts, or stops after completing its task.

During analysis, investigators try to identify evidence from each stage. For example, an email attachment indicates delivery, a new process indicates execution, a Run registry key indicates persistence, and an unknown domain may indicate C2.

Conclusion: Understanding the lifecycle helps defenders detect malware at multiple stages instead of relying on only one detection method.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 4

Q1. Analyze the given executable using Detect It Easy (DIE) and identify the compiler, packer, architecture, and entropy. Prepare a report. [15M]

Answer

Aim: To identify basic executable characteristics and signs of packing using Detect It Easy.

Tools Required: Detect It Easy (DIE) and the given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Open Detect It Easy and load the executable.
2. Check the detected file format, such as PE32 or PE32+.
3. Note the architecture, for example x86 (32-bit) or x64 (64-bit).
4. Record the compiler/linker signature if DIE identifies one, such as Microsoft Visual C/C++, MinGW, Delphi, etc.
5. Check whether DIE detects a packer/protector such as UPX or another known packing signature.
6. Open the Entropy view and note the overall or section entropy values.
7. High entropy, especially close to the upper range, can indicate compression/encryption/packing, but it is not proof by itself.
8. Check section names and sizes for additional anomalies.
9. Take screenshots showing detected architecture/compiler/packer and entropy.
10. Write a short conclusion describing whether packing is detected or only suspected.

What to Observe / Record

- File format
- Architecture
- Compiler/linker
- Packer/protector
- Entropy
- Unusual sections

Report Format

Field	Value
File name	_____
Format	_____
Architecture	_____
Compiler	_____
Packer / Protector	_____
Entropy	_____
Packing verdict	Detected / Suspected / Not detected

How to generate the report: Copy the exact DIE detections into the report. If DIE shows "unknown" for compiler or packer, write unknown rather than guessing. Explain entropy as supporting evidence only.

Result / Conclusion: The executable architecture, probable compiler, packing information, and entropy were identified using DIE.

Q2. Explain the objectives of malware analysis. [15M]

Answer

The main objective of malware analysis is to understand what a suspicious program is, what it does, how it affects a system, and how defenders can detect and remove it safely.

- Identify whether a file is malicious, suspicious, or benign.
- Determine malware type or family, such as Trojan, ransomware, spyware, worm, downloader, or backdoor.
- Understand behavior: processes created, files modified, registry changes, persistence, network activity, and other actions.

- Extract Indicators of Compromise (IOCs) such as hashes, file names, domains, IP addresses, registry keys, mutexes, or URLs.
- Discover the infection method and how the malware enters or executes on the system.
- Identify persistence mechanisms that allow malware to survive reboot or user logoff.
- Understand network communication, including command-and-control or data exfiltration attempts.
- Support containment and incident response by identifying affected hosts and blocking known indicators.
- Improve security controls by creating antivirus/EDR detections, YARA/Sigma rules, firewall blocks, or monitoring logic where appropriate.
- Help remove the malware and recover the system safely.
- Assess impact, such as stolen credentials, encrypted files, damaged data, or unauthorized access.
- Provide evidence and a clear technical report for administrators, incident responders, or management.

Conclusion: Malware analysis converts an unknown suspicious file into actionable information that can be used for detection, containment, remediation, and prevention.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 5

Q1. Analyze the Portable Executable using PView and identify the DOS Header, PE Header, Entry Point, and Sections. [15M]

Answer

Aim: To inspect the internal structure of a Windows PE executable using PView.

Tools Required: PView and the given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Open PView and load the executable.
2. Expand IMAGE_DOS_HEADER and verify the DOS signature (MZ). Note e_lfanew, which points to the PE header.
3. Open the PE/NT Headers and verify the PE signature.
4. Inspect IMAGE_FILE_HEADER / COFF header and note Machine type, NumberOfSections, and TimeDateStamp if required.
5. Open IMAGE_OPTIONAL_HEADER and record AddressOfEntryPoint, ImageBase, Subsystem, and other important values.
6. Expand the Section Headers and list section names such as .text, .data, .rdata, .rsrc, .reloc, .idata, etc.
7. For each important section, note virtual size, raw size, and characteristics/permissions if shown.
8. Compare the entry point with the section ranges to identify which section contains it.
9. Look for unusual section names, strange sizes, or suspicious permissions.
10. Take screenshots of DOS header, PE header/entry point, and section table.

What to Observe / Record

- MZ signature
- e_lfanew
- PE signature
- Machine type
- Number of sections
- AddressOfEntryPoint
- ImageBase
- Section names/sizes/permissions

Report Format

Item	Observed value
DOS signature	_____
e_lfanew	_____
PE signature	_____
Machine	_____
Number of sections	_____
Entry point	_____
ImageBase	_____
Sections	_____

How to generate the report: Use the actual hexadecimal values shown by PView. If needed, add a small table for each section with Name, Virtual Size, Raw Size, and Characteristics.

Result / Conclusion: The PE file structure was examined and the DOS header, PE headers, entry point, and sections were identified.

Q2. Explain the basic structure of a PE file. [15M]

Answer

PE stands for Portable Executable. It is the standard Windows format used for EXE, DLL, SYS and similar executable files. A PE file contains headers that describe how Windows should load it, followed by sections containing code and data.

DOS Header -> DOS Stub -> PE Signature -> COFF Header -> Optional Header -> Section Table -> Sections

- DOS Header: Starts with the MZ signature. The e_lfanew field points to the location of the PE header.
- DOS Stub: A small legacy program usually containing a message such as "This program cannot be run in DOS mode."
- PE Signature: The four-byte signature "PE\0\0" confirms the PE structure.
- COFF/File Header: Contains machine architecture, number of sections, timestamp, and other file-level information.
- Optional Header: Despite the name, it is required for normal executable images. It contains entry point, image base, alignment, subsystem, sizes, and data directory locations.
- Data Directories: Pointers to important structures such as Import Table, Export Table, Resource Table, Relocation Table, Debug information, and TLS.
- Section Table: Describes each section, including its name, virtual address, raw offset, size, and characteristics.
- .text: Usually contains executable program code.
- .data: Usually stores initialized writable data.
- .rdata: Usually contains read-only data and constants.
- .idata: Commonly related to imported functions/DLLs.
- .rsrc: Stores resources such as icons, dialogs, menus, and version information.
- .reloc: Contains relocation information used when the image cannot load at its preferred base address.

Conclusion: PE analysis is important because malware often leaves clues in headers, imports, entry point, section names, permissions, and section entropy.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 6

Q1. Calculate the MD5 and SHA-256 hash values of the given executable and verify the file using VirusTotal. Prepare a report. [15M]

Answer

Aim: To uniquely identify the executable using cryptographic hashes and check the same file against VirusTotal.

Tools Required: Windows certutil / PowerShell Get-FileHash / hashing utility, VirusTotal, given executable.

Safe Setup: Follow lab policy before uploading a sample. Searching by SHA-256 is preferred when the file is already known to VirusTotal.

Step-by-Step Procedure

1. Open Command Prompt or PowerShell in the folder containing the sample.
2. Calculate MD5, for example: `certutil -hashfile sample.exe MD5` . Record the complete output hash.
3. Calculate SHA-256, for example: `certutil -hashfile sample.exe SHA256` .
4. Alternatively, PowerShell can be used with `Get-FileHash -Algorithm MD5` and `Get-FileHash -Algorithm SHA256`.
5. Copy the SHA-256 hash exactly and search it on VirusTotal.
6. If no result exists and lab policy permits, upload the sample for scanning.
7. Note the detection ratio and compare the hash shown by VirusTotal with your locally calculated hash.
8. Record file type and 3-5 vendor results.
9. Take screenshots of the local hash output and VirusTotal result.
10. Write a conclusion that the file identity was verified when the local and VirusTotal hash values match.

What to Observe / Record

- MD5
- SHA-256
- VirusTotal detection ratio
- Matching hash confirmation
- File type
- Selected vendor verdicts

Report Format

Field	Value
File name	_____
MD5	_____
SHA-256	_____
VirusTotal detection ratio	____ / ____
File type	_____
Hash match	Yes / No
Final verdict	_____

How to generate the report: Show both locally calculated hashes first, then VirusTotal results. The most important verification point is that the SHA-256 searched on VirusTotal matches the sample you hashed.

Result / Conclusion: MD5 and SHA-256 values were calculated and the file was checked on VirusTotal using its hash.

Q2. Explain the importance of file hashing in malware analysis. [15M]

Answer

A file hash is a fixed-length value calculated from file contents. Common algorithms used in malware analysis include MD5, SHA-1, and especially SHA-256. Even a very small change in the file usually produces a different hash.

- Unique identification: Analysts can refer to a sample by hash instead of relying only on its file name, which can easily be changed.
- Integrity checking: A hash shows whether the file has changed between collection, transfer, and analysis.
- VirusTotal / threat intelligence lookup: Hashes can be searched to find previous detections, malware family names, or known reports.
- IOC creation: A malicious hash can be used as an Indicator of Compromise for scanning endpoints or logs.

- Deduplication: Large malware collections can identify exact duplicate samples using hashes.
- Chain of custody: In forensic work, hashes help prove that evidence remained unchanged.
- Incident response: Security teams can search across systems for a known malicious hash.
- Automation: Malware analysis platforms use hashes to index and correlate samples.
- Comparison: If two files have the same strong cryptographic hash, they are effectively treated as identical copies for practical analysis.
- Limitation: Attackers can change even one byte to create a new hash, so hash-based detection alone cannot detect all variants.

MD5 is still useful for quick sample identification but is not collision-resistant for security purposes. SHA-256 is preferred when a stronger cryptographic identifier is needed.

Conclusion: Hashing is simple but essential because it provides a reliable identity for a malware sample and supports integrity, threat intelligence, IOC matching, and evidence handling.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 7

Q1. Compare the results obtained from VirusTotal and PESTudio for the given executable. Prepare a report. [15M]

Answer

Aim: To compare online multi-engine detection with local static PE analysis for the same executable.

Tools Required: VirusTotal, PESTudio, given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Calculate or note the sample SHA-256 so both tools clearly refer to the same file.
2. Analyze/search the sample on VirusTotal and record detection ratio, file type, hashes, important vendor names, and available IOCs.
3. Open the same file in PESTudio and record file information, imports, suspicious APIs, indicators, strings, and section anomalies.
4. Compare the hash values to confirm the same sample is being analyzed.
5. Compare VirusTotal vendor detections with PESTudio suspicious indicators.
6. Note evidence available only in VirusTotal, such as multi-engine reputation or previous submissions.
7. Note evidence available only in PESTudio, such as detailed imports, PE structure, strings, or indicators.
8. Identify findings that support each other, for example a network-related API in PESTudio and a malicious network IOC in VirusTotal.
9. Prepare a comparison table and attach screenshots from both tools.
10. Write a combined conclusion instead of relying on one tool alone.

What to Observe / Record

- Same file hash
- VirusTotal detection ratio
- PESTudio suspicious indicators
- Imports and APIs
- Strings/IOCs
- Agreement or disagreement between tools

Report Format

Aspect	VirusTotal	PEStudio
Main purpose	Multi-engine/reputation scan	Static PE inspection
Hash	_____	_____
Malicious detections	_____	N/A / indicators
Imports/APIs	Limited/varies	_____
Strings	May show some	_____
IOCs	_____	_____
Final interpretation	_____	_____

How to generate the report: Use one combined table, then add 3-5 bullet points explaining where both tools agree and where they provide different information.

Result / Conclusion: VirusTotal and PESTudio results were compared to obtain a more complete static assessment of the same executable.

Q2. Explain Static Malware Analysis and Dynamic Malware Analysis. [15M]

Answer

Malware analysis is commonly divided into static and dynamic analysis. Static analysis studies a file without executing it, while dynamic analysis observes what the file does during controlled execution.

- Static analysis usually begins with hashes, file type, strings, PE headers, imports, sections, and packer detection.
- Dynamic analysis monitors process creation, file writes, registry activity, network communication, and persistence during execution.

- Both methods should be combined because each reveals information the other may miss.

Point	Static Analysis	Dynamic Analysis
Execution	File is not executed.	File is executed in an isolated environment.
Typical tools	VirusTotal, PEStudio, PEview, DIE, Strings, disassemblers.	Process Monitor, Process Explorer, Wireshark, sandbox tools, debuggers.
Findings	Hashes, file type, PE headers, imports, strings, packer signs.	Processes, files created, registry changes, persistence, network traffic.
Risk	Lower because sample is not run.	Higher; requires VM/sandbox isolation.
Strength	Fast and good for initial triage.	Shows actual runtime behavior.
Limitation	Packing/obfuscation may hide details.	Malware may detect sandbox/VM or need special triggers.

Conclusion: Static analysis is safer and faster for initial inspection, while dynamic analysis confirms real behavior. A strong malware investigation usually uses both.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 8

Q1. Explain how to create a safe malware analysis environment using VMware or VirtualBox. [15M]

Answer

Aim: To prepare an isolated virtual machine environment where suspicious samples can be analyzed without exposing the host system or personal network.

Tools Required: VMware Workstation/Player or Oracle VirtualBox, Windows analysis VM, malware analysis tools.

Safe Setup: Use only authorized lab samples. Do not use your normal personal machine as the malware execution environment.

Step-by-Step Procedure

1. Install VMware or VirtualBox on the host computer.
2. Create a new virtual machine and install a fresh Windows operating system for analysis.
3. Install required analysis tools such as PESTudio, DIE, Strings, Process Explorer, ProcMon, Wireshark, and any lab-approved utilities.
4. Disable shared folders, shared clipboard, drag-and-drop, USB auto-connect, and unnecessary host integration features before running malware.
5. Configure networking safely. For pure static analysis, keep networking disabled. For dynamic network analysis, use an isolated/host-only or lab-controlled network as instructed by the examiner.
6. Do not sign in to personal accounts or store personal files in the analysis VM.
7. Create a clean snapshot named, for example, "Clean-Before-Analysis".
8. Copy the sample into the VM using the lab-approved method and keep it password-protected if supplied that way.
9. Perform analysis and collect screenshots/logs.
10. After the practical, power off the VM and revert to the clean snapshot to remove changes made by the sample.

What to Observe / Record

- VM isolation settings
- Network mode
- Snapshot name
- Installed analysis tools
- Host integration features disabled

Report Format

Item	Configuration
Hypervisor	VMware / VirtualBox
Guest OS	_____
Network mode	Disabled / Host-only / Lab-controlled
Shared folders	Disabled
Clipboard / Drag-drop	Disabled
Snapshot	Clean snapshot created
Tools installed	_____

How to generate the report: Write the configuration table, attach a screenshot of VM network/settings and snapshot, then state why each control reduces risk.

Result / Conclusion: A controlled malware analysis VM was created with isolation controls and a clean snapshot for safe rollback.

Q2. Explain the purpose of sandboxing in malware analysis. [15M]

Answer

A sandbox is an isolated environment used to execute suspicious files while monitoring their behavior. The goal is to observe malware without allowing it to affect the real production system.

- Isolation: Keeps suspicious activity separated from the analyst's real computer and important data.
- Behavior observation: Shows processes, files, registry keys, services, scheduled tasks, and other runtime actions.

- Network observation: Can reveal DNS requests, IP connections, URLs, protocols, and command-and-control behavior.
- Automated analysis: Online sandboxes can automatically generate behavior reports and IOCs.
- Persistence discovery: Helps identify how malware tries to survive reboot or logoff.
- Dropped-file discovery: Records additional files created or downloaded by the sample.
- IOC extraction: Produces hashes, file names, domains, IPs, registry paths, and other indicators useful for detection.
- Fast triage: Analysts can quickly decide whether a file needs deeper manual analysis.
- Repeatability: Snapshots or disposable environments allow the same test to be repeated.
- Limitation: Some malware detects VMs/sandboxes, delays execution, or needs user interaction/network conditions, so sandbox results are not always complete.

Conclusion: Sandboxing reduces risk and provides valuable runtime evidence, but the results should be combined with static analysis and analyst judgment.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 9

Q1. Analyze a suspicious executable using PESTudio and identify imported DLLs and suspicious APIs. [15M]

Answer

Aim: To inspect imported libraries and Windows API functions that may indicate suspicious capabilities.

Tools Required: PESTudio and the given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Open the executable in PESTudio.
2. Go to Libraries and list the imported DLLs.
3. Identify normal core DLLs such as KERNEL32.dll, USER32.dll, ADVAPI32.dll and note networking DLLs such as WININET.dll, WINHTTP.dll, WS2_32.dll if present.
4. Open Imports and review the imported functions.
5. Mark process-related APIs such as CreateProcess, OpenProcess, CreateRemoteThread, or similar functions if present.
6. Mark memory-related APIs such as VirtualAlloc, VirtualProtect, WriteProcessMemory, etc. if present.
7. Mark registry/service APIs such as RegSetValue, OpenSCManager, CreateService, etc. if present.
8. Mark networking/download APIs such as InternetOpen, InternetConnect, URLDownloadToFile, socket, connect, send, recv, etc. if present.
9. Check PESTudio Indicators to see whether any of those imports are flagged.
10. Prepare a table that lists each suspicious API and a possible legitimate/malicious use, then write a balanced conclusion.

What to Observe / Record

- Imported DLL names
- Suspicious API names
- Category of capability
- PESTudio indicator score/flag

Report Format

DLL / API	Category	Why it may be suspicious
_____	Process / Memory / Registry / Network	_____
_____	_____	_____
_____	_____	_____

How to generate the report: Do not call an API malicious only because it appears in imports. Write "may indicate" and support the conclusion using multiple findings such as strings, indicators, or reputation.

Result / Conclusion: Imported DLLs and APIs were identified and categorized to understand possible capabilities of the executable.

Q2. Explain the role of Strings and PEiD in static malware analysis. [15M]

Answer

Strings and PEiD are basic static-analysis tools. They help an analyst learn about an executable without running it.

- Strings extracts readable ASCII/Unicode text embedded in a binary file.
- Useful strings may include URLs, IP addresses, domain names, file paths, registry keys, commands, mutex names, error messages, user-agent strings, and malware configuration text.
- String analysis can quickly reveal possible network destinations or persistence locations.
- Limitation of Strings: packed or encrypted malware may hide most useful text, and many extracted strings may be meaningless.
- PEiD is mainly used to identify packers, cryptors, protectors, and compiler signatures in Windows PE files.
- PEiD compares the file's entry point and signatures against a database of known patterns.

- Detecting a packer is useful because packed code often needs unpacking or dynamic analysis before deeper inspection.
- PEiD can also provide basic PE information and hints about how the file was built.
- Limitation of PEiD: its signature database can be outdated, custom packers may not be recognized, and a detection is not proof of maliciousness.
- Together, Strings provides readable content while PEiD provides build/packing information, making both useful for initial triage.

Conclusion: Strings answers "what readable clues are inside?" while PEiD helps answer "how was this PE built or packed?" Both are useful but should be combined with other tools.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 10

Q1. Use Detect It Easy (DIE) to determine whether the executable is packed. Prepare a report. [15M]

Answer

Aim: To determine whether a Windows executable is compressed/protected with a packer using DIE.

Tools Required: Detect It Easy and the given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Open the executable in Detect It Easy.
2. Note the file format and architecture.
3. Check the main detection/signature panel for packer or protector names.
4. If DIE identifies a known packer, record the exact name and version if shown.
5. Open entropy information. High entropy can support a packing suspicion because packed/encrypted data appears more random.
6. Check section names. Packers may create unusual section names or very small/large sections, although this varies.
7. Compare raw size and virtual size if DIE provides section details.
8. Look for a small import table containing only basic loader functions, which can also suggest packing.
9. Take screenshots of signature detection and entropy/section information.
10. Conclude "packed", "likely packed", or "no clear evidence of packing" based on actual evidence.

What to Observe / Record

- Detected packer/protector
- Architecture
- Entropy
- Section anomalies
- Import-table size

Report Format

Field	Finding
File	_____
Architecture	_____
Packer signature	_____
Entropy	_____
Section signs	_____
Final packing verdict	Packed / Suspected / Not detected

How to generate the report: The report should separate direct evidence (DIE signature) from indirect evidence (high entropy, unusual sections). If no packer is detected, do not claim one based only on entropy.

Result / Conclusion: The executable was checked for packing using DIE and the conclusion was recorded from signature and structural evidence.

Q2. Explain packed and obfuscated malware. [15M]

Answer

Malware authors often use packing and obfuscation to make analysis and detection more difficult. Both techniques hide the real code, but they work in different ways.

- Legitimate software can also use packers/protectors, so packing alone does not prove maliciousness.
- Obfuscation can be simple, such as Base64/XOR encoded strings, or complex, such as control-flow flattening and runtime API resolution.
- Packed and obfuscated malware often requires both static and dynamic analysis because important content is hidden before execution.

Point	Packing	Obfuscation
Meaning	Compresses/encrypts the original	Changes code or data representation so

	program and adds a small unpacking routine.	it is difficult to understand while preserving behavior.
Runtime	Original code is usually restored/unpacked in memory before execution.	Obfuscated code may execute directly but is intentionally hard to read/analyze.
Common signs	High entropy, unusual sections, small import table, known packer signature.	Meaningless names, encoded strings, control-flow tricks, junk code, API hashing.
Purpose	Reduce size or hide the original executable; malware uses it mainly to evade analysis/detection.	Hide logic, strings, configuration, and behavior from analysts or security tools.
Analysis response	Identify packer, unpack/decompress if possible, then analyze original code.	Decode/deobfuscate strings or logic and use dynamic analysis to reveal behavior.

Conclusion: Packing mainly hides the executable by wrapping/compressing it, while obfuscation makes the internal logic difficult to understand. Both are common anti-analysis techniques.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 11

Q1. Monitor the execution of a malware sample using Process Explorer and identify suspicious processes. [15M]

Answer

Aim: To observe processes created by a suspicious sample and identify abnormal process behavior.

Tools Required: Process Explorer (Sysinternals), isolated analysis VM, given sample.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Revert the analysis VM to a clean snapshot and disconnect/contain networking according to lab instructions.
2. Open Process Explorer as administrator before running the sample.
3. Note the baseline processes currently running.
4. Execute the sample only inside the isolated VM.
5. Watch the process tree for a new parent process and any child processes it creates.
6. For suspicious processes, open Properties and record image path, command line, parent PID, company/signature information, start time, and loaded DLLs if useful.
7. Look for unusual behavior such as a process running from Temp/AppData, random file names, unexpected child shells, or unsigned binaries in unusual locations.
8. Check CPU, memory, handles, and network-related indicators shown by Process Explorer where relevant.
9. Take screenshots of the process tree and properties of suspicious processes.
10. Stop the experiment as instructed and revert the VM snapshot after collecting evidence.

What to Observe / Record

- Process name and PID
- Parent-child relationship
- Image path
- Command line
- Digital signature/company
- Unexpected child processes
- Resource usage

Report Format

Process	PID / Parent	Path / Command line	Why suspicious
_____	_____	_____	_____
_____	_____	_____	_____

How to generate the report: Start with baseline vs after-execution comparison. Include one process-tree screenshot and one properties screenshot. Describe observed facts first, then explain why they are suspicious.

Result / Conclusion: Process Explorer was used to monitor runtime process activity and suspicious process relationships were documented.

Q2. Explain behavior analysis in malware analysis. [15M]

Answer

Behavior analysis studies what a suspicious program actually does when it runs in a controlled environment. It is a form of dynamic analysis.

- Process behavior: Records new processes, parent-child relationships, command lines, and process injection indicators.
- File-system behavior: Observes files created, modified, renamed, or deleted.
- Registry behavior: Detects persistence settings, configuration changes, and security-related modifications.
- Persistence behavior: Finds startup entries, services, scheduled tasks, startup-folder changes, or other mechanisms used to survive reboot.
- Network behavior: Captures DNS requests, IP connections, URLs, protocols, and possible C2 traffic.
- Memory behavior: Can reveal unpacked code, injected modules, or configuration that is not visible in the static file.

- User/system changes: May show disabled security tools, changed settings, credential access attempts, or data collection.
- Tools: Process Explorer, ProcMon, Wireshark, Regshot, sandboxes, debuggers, and memory analysis tools are commonly used.
- Advantage: It reveals real actions even when the file is packed or obfuscated.
- Limitation: Malware may detect sandboxes, wait for user actions, use time delays, or require internet/server availability.

Conclusion: Behavior analysis confirms what the malware actually does, making it important for IOC extraction, impact assessment, and incident response.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 12

Q1. Use Process Monitor (ProcMon) to identify file and registry changes after executing a sample. Prepare a report. [15M]

Answer

Aim: To monitor file-system and registry operations generated by a suspicious sample.

Tools Required: Process Monitor (Sysinternals), isolated analysis VM, given sample.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Revert the VM to a clean state and start Process Monitor before executing the sample.
2. Clear old events using the Clear Display option so the capture begins from a clean view.
3. Add a filter for Process Name is sample.exe (or the actual process name). Include child processes if they are important.
4. Start capturing events.
5. Execute the sample in the isolated VM.
6. Observe file operations such as CreateFile, WriteFile, SetDispositionInformation/Delete, Rename, and directory creation.
7. Observe registry operations such as RegCreateKey, RegSetValue, RegDeleteValue, and access to Run/RunOnce or service-related keys.
8. Use filters to focus on successful writes/changes and important paths rather than thousands of read operations.
9. Save the ProcMon capture if required and take screenshots of key file and registry events.
10. Write a report listing each important change and its possible purpose, then revert the VM.

What to Observe / Record

- New/dropped files
- Modified/deleted files
- Registry keys created/modified
- Persistence-related keys
- Child-process operations
- Operation result (SUCCESS/NAME NOT FOUND, etc.)

Report Format

Time	Process	Operation	Path	Result / Interpretation
_____	_____	WriteFile / RegSetValue	_____	_____
_____	_____	_____	_____	_____

How to generate the report: Filter first, then export or screenshot only meaningful events. Group report findings into File Changes and Registry Changes, and finish with any persistence-related finding.

Result / Conclusion: ProcMon captured the sample's file-system and registry activity and the important changes were documented.

Q2. Explain the importance of ProcMon in malware analysis. [15M]

Answer

Process Monitor (ProcMon) is a Sysinternals tool that records real-time file-system, registry, process/thread, and related system activity. It is very useful during dynamic malware analysis.

- Reveals dropped files and locations used by malware.
- Shows file creation, writes, deletes, renames, and access patterns.
- Records registry keys and values created or modified by the sample.
- Helps detect persistence through Run keys, services, or other startup-related changes.
- Shows which process performed each operation and the exact path involved.
- Captures timestamps and operation results, helping reconstruct the sequence of behavior.
- Powerful filtering allows the analyst to isolate one process, operation type, or path.

- Can identify configuration discovery, DLL loading, temporary files, and attempts to access sensitive locations.
- Logs can be saved for later examination and included as evidence in a malware report.
- Limitation: ProcMon generates a very large number of events, so good filtering and analyst interpretation are essential.

Conclusion: ProcMon is important because it provides detailed evidence of how malware interacts with Windows files and registry, which helps identify behavior and persistence.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 13

Q1. Capture malware-generated network traffic using Wireshark and identify DNS requests and IP addresses. [15M]

Answer

Aim: To capture network activity generated by a suspicious sample and identify DNS queries and communicating IP addresses.

Tools Required: Wireshark, isolated/lab-controlled network, analysis VM, given sample.

Safe Setup: Use only a lab-approved network configuration. Do not allow an unknown sample uncontrolled access to the normal personal/production network.

Step-by-Step Procedure

1. Start Wireshark inside the analysis environment and select the correct network interface.
2. Start packet capture before executing the sample.
3. Execute the sample in the isolated/lab-controlled environment.
4. After sufficient activity is captured, stop the capture.
5. Apply the display filter dns to view DNS traffic.
6. Identify domain names queried by the sample and note corresponding response IP addresses when available.
7. Use filters such as ip.addr == x.x.x.x, tcp, http, tls, or other relevant protocols to inspect connections.
8. Use Statistics > Endpoints / Conversations to list communicating IP addresses and connection pairs.
9. Correlate timestamps with the time the sample was executed so unrelated background traffic is not mistaken for malware traffic.
10. Save the PCAP and take screenshots of important DNS and IP findings.

What to Observe / Record

- DNS query names
- DNS response IPs
- Source and destination IPs
- Ports
- Protocols
- Repeated or suspicious connections

Report Format

Time	DNS / IP	Port/Protocol	Observation
_____	domain.example / x.x.x.x	DNS / TCP / TLS	_____
_____	_____	_____	_____

How to generate the report: List only traffic that can reasonably be associated with the sample based on time/process/lab setup. Attach a screenshot and, if required, the saved PCAP file.

Result / Conclusion: Wireshark was used to capture the sample's network activity and identify relevant DNS requests and IP addresses.

Q2. Explain network activity analysis in malware investigation. [15M]

Answer

Network activity analysis studies the communication generated by a suspicious program or infected host. It helps reveal external infrastructure and the purpose of malware communication.

- DNS analysis can reveal domains contacted by malware and their resolved IP addresses.
- IP/port analysis shows remote systems and services the malware communicates with.
- Protocol analysis identifies HTTP, HTTPS/TLS, DNS, TCP, UDP, SMB, or other protocols used.
- Command-and-control detection: Repeated beaconing or unusual outbound connections can suggest C2 communication.
- Download behavior: Traffic may show the malware downloading additional payloads or configuration.
- Data exfiltration clues: Unusual outbound transfers may indicate stolen data leaving the host.
- IOC extraction: Domains, IPs, URLs, ports, and certificates can become network IOCs.
- Correlation: Network timestamps can be matched with process, file, and registry events from other tools.
- Tools: Wireshark, tcpdump, Zeek, IDS/IPS logs, firewall logs, proxy logs, and sandbox network reports.

- Limitations: Encryption can hide content, shared IPs/CDNs may create false assumptions, and background traffic must be separated from malware traffic.

Conclusion: Network analysis is important because it can expose malware infrastructure and communication patterns that are not visible from the file alone.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 15

Q1. Identify Indicators of Compromise (IOCs) from a VirusTotal or Hybrid Analysis report. [15M]

Answer

Aim: To extract useful Indicators of Compromise from an existing malware-analysis report.

Tools Required: VirusTotal or Hybrid Analysis report provided in the practical.

Safe Setup: Use the report supplied by the examiner; do not invent indicators that are not present.

Step-by-Step Procedure

1. Open the VirusTotal or Hybrid Analysis report for the sample.
2. Record file hashes such as MD5, SHA-1, and SHA-256.
3. Check for file names, dropped files, created files, or paths related to the sample.
4. Check network information for URLs, domain names, IP addresses, and ports.
5. Check behavior/registry sections for registry keys or values created or modified.
6. Check process information for suspicious process names, command lines, mutexes, services, or scheduled tasks if reported.
7. Classify each IOC by type instead of writing one long mixed list.
8. Where possible, include context: what created the IOC or why it is related to the sample.
9. Avoid using common legitimate values as strong IOCs without context.
10. Prepare a final IOC table that can be used by an incident responder for searching or blocking.

What to Observe / Record

- Hashes
- File names/paths
- Domains/URLs
- IP addresses/ports
- Registry keys
- Processes/commands
- Mutex/service/task names

Report Format

IOC Type	IOC Value	Context
SHA-256	_____	Sample hash
Domain / URL	_____	Contacted by sample
IP address	_____	Network connection
File path	_____	Created/dropped file
Registry key	_____	Persistence/configuration
Process/Command	_____	Observed behavior

How to generate the report: Copy only indicators that actually appear in the report. Group them by type, include one-line context, and finish with which IOCs are strongest for threat hunting.

Result / Conclusion: Relevant host and network IOCs were extracted and organized from the malware-analysis report.

Q2. Explain Indicators of Compromise with examples. [15M]

Answer

Indicators of Compromise (IOCs) are observable pieces of evidence that may show a system has been attacked or infected. Security teams use IOCs to search logs, detect related systems, block known threats, and support incident response.

- File hashes: MD5/SHA-256 values of known malicious files. Example: a SHA-256 matching a confirmed malware sample.
- Malicious file names or paths: Example: an unexpected executable created in AppData or Temp.
- Domain names: A domain contacted by malware for command-and-control or downloads.
- IP addresses: A remote IP repeatedly contacted by an infected host.
- URLs: A specific malicious download or callback URL.

- Registry keys/values: Startup entries created for persistence.
- Processes/command lines: Unusual commands or child-process patterns associated with an attack.
- Mutex names: Some malware creates unique mutex values to avoid running multiple copies.
- Services or scheduled tasks: New entries created to maintain persistence.
- Email indicators: Sender address, attachment hash, subject, or malicious link in a phishing incident.

IOCs can be fragile. Hashes are very specific but change when a file changes. Domains and IPs may also change or be shared by legitimate services. Therefore, IOCs should be used with context and combined with behavior-based detection.

Conclusion: IOCs are practical evidence for detecting and investigating compromise, but they are strongest when correlated with behavior and multiple data sources.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 16

Q1. Analyze the imported functions of an executable using PESTudio and identify suspicious Windows API calls. [15M]

Answer

Aim: To analyze imported Windows functions and infer possible capabilities of the executable.

Tools Required: PESTudio and the given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Open the sample in PESTudio and confirm its file information.
2. Open Libraries and Imports.
3. List important imported DLLs and then inspect individual API names.
4. Group APIs into categories: Process, Memory, File, Registry, Network, Service/Persistence, Cryptography, and User/Input.
5. Examples of process/memory APIs to note if present include OpenProcess, VirtualAlloc/VirtualAllocEx, WriteProcessMemory, CreateRemoteThread, VirtualProtect, etc.
6. Examples of registry/service APIs include RegCreateKey, RegSetValue, OpenSCManager, CreateService, StartService, etc.
7. Examples of network APIs include InternetOpen, WinHttpOpen, socket, connect, send, recv, URLDownloadToFile, etc.
8. Check PESTudio indicator highlighting and strings for supporting evidence.
9. Write what each suspicious API can do, but avoid claiming malicious behavior unless other evidence supports it.
10. Prepare a categorized table and final capability summary.

What to Observe / Record

- Imported DLL
- API name
- Capability category
- PESTudio warning/indicator
- Supporting strings or other evidence

Report Format

API	DLL	Category	Possible meaning
_____	_____	Process / Memory / Registry / Network	_____
_____	_____	_____	_____

How to generate the report: A good report explains capability, not guilt. For example: "WriteProcessMemory may support process manipulation; further dynamic analysis is required to confirm its use."

Result / Conclusion: Imported functions were reviewed and suspicious Windows APIs were identified and categorized.

Q2. Explain the importance of imported functions in malware analysis. [15M]

Answer

Imported functions show which operating-system services an executable may call. Because Windows programs use APIs to access files, processes, registry, network, memory, and services, the import table gives useful clues about possible behavior.

- Capability inference: Imports can indicate whether a program can create processes, access files, modify registry, connect to networks, or use cryptography.
- Persistence clues: Service and registry APIs may suggest startup or persistence capabilities.
- Process manipulation clues: APIs for opening processes, allocating memory, or writing memory can indicate advanced behavior when combined with other evidence.
- Network clues: Internet or socket APIs can reveal that a program has communication capability.
- File activity clues: CreateFile, WriteFile, DeleteFile and similar APIs suggest file-system operations.

- Cryptographic clues: Crypto APIs may be relevant to ransomware, protected configuration, or normal legitimate security functions.
- Anti-analysis clues: Some APIs can query system information, timing, or debugging state.
- Prioritization: Suspicious imports help the analyst decide which areas need dynamic analysis.
- Context is essential: Many suspicious-looking APIs are also used by legitimate software, so imports alone do not prove maliciousness.
- Packed malware limitation: A packed file may have a very small import table and resolve APIs dynamically, so static imports may not show all capabilities.

Conclusion: Imported functions are valuable for predicting possible behavior, but they must be interpreted with strings, PE structure, reputation, and dynamic evidence.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 17

Q1. Perform string extraction and classify the extracted strings into URLs, registry keys, file paths, and commands. [15M]

Answer

Aim: To extract readable strings from the executable and organize them into meaningful IOC/behavior categories.

Tools Required: Strings utility and text editor/grep/search function.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Run the Strings utility on the sample and redirect output to a text file.
2. Open the output and remove/ignore obvious noise and very short meaningless strings.
3. Search for http://, https://, www, domain extensions, and IP-like patterns to identify URLs/network strings.
4. Search for HKLM, HKCU, Software\, CurrentVersion\Run and similar patterns for registry keys.
5. Search for drive letters, AppData, Temp, System32, ProgramData and executable/script extensions for file paths.
6. Search for cmd.exe, powershell, rundll32, reg, schtasks, wmic, mshta, curl or other command-related text.
7. Copy each useful string exactly as it appears.
8. Classify it into URL, Registry Key, File Path, or Command.
9. Add a short interpretation explaining what the string may indicate.
10. Prepare the final table and include a screenshot or text output as evidence.

What to Observe / Record

- Network strings
- Registry strings
- File paths
- Command strings
- Other meaningful text

Report Format

Category	Extracted string	Interpretation
URL	_____	Possible communication/download
Registry Key	_____	Possible configuration/persistence
File Path	_____	Possible file location
Command	_____	Possible execution command

How to generate the report: Classify strings into the four required categories first. Then explain only the important ones. Do not treat a string as proof that the malware actually used it; dynamic analysis may be needed.

Result / Conclusion: Strings were extracted and classified into URL, registry, file-path, and command categories.

Q2. Explain the importance of string analysis. [15M]

Answer

String analysis is one of the fastest static-analysis techniques. It extracts readable text from a binary and can reveal clues about program behavior without executing the file.

- Finds URLs, domains, and IP addresses that may be used for command-and-control or downloads.
- Reveals file paths and file names used by the program.
- Reveals registry keys that may be used for configuration or persistence.
- Finds command lines, PowerShell/CMD commands, script names, and system utilities.
- Can expose user-agent strings, HTTP paths, email addresses, error messages, and debug information.
- May reveal malware family names, campaign identifiers, mutex names, service names, or configuration data.
- Helps generate IOCs quickly for threat hunting.
- Helps guide later analysis by showing which functions or behaviors deserve attention.
- Works without executing the sample, so it is low-risk compared with dynamic analysis.
- Limitation: Packed, encrypted, or obfuscated malware may hide strings; extracted text may also include false positives or unused data.

Conclusion: String analysis is a simple and powerful first step, but important findings should be verified with other static or dynamic evidence.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 18

Q1. Compare the outputs of PEview and PESTudio for the same executable. [15M]

Answer

Aim: To compare a low-level PE structure viewer with a higher-level static malware triage tool.

Tools Required: PEview, PESTudio, same executable file.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Open the sample in PEview and record DOS header, PE/COFF header, optional header, entry point, and section details.
2. Open the same sample in PESTudio and record file information, hashes, imports, sections, strings, and indicators.
3. Confirm both tools show the same file architecture, entry point, number of sections, and basic PE information where applicable.
4. Compare section names and sizes reported by both tools.
5. Note that PEview presents raw/low-level structures and hexadecimal fields, while PESTudio adds automated interpretation and suspicious indicators.
6. Compare import information if both expose it.
7. Use PEview to verify exact header values and PESTudio to highlight suspicious characteristics.
8. Take one screenshot from each tool showing the same or related information.
9. Prepare a side-by-side comparison table.
10. Write which tool is more useful for structure inspection and which is faster for malware triage.

What to Observe / Record

- Architecture
- Entry point
- Number of sections
- Section names/sizes
- Imports
- Automated indicators
- Raw header detail

Report Format

Feature	PEview	PEStudio
Main focus	Raw PE structure	Static malware triage
Headers	Detailed low-level values	Summarized/parsed
Entry point	_____	_____
Sections	_____	_____
Imports	Available / basic	Detailed with indicators
Strings	Limited / not main feature	Integrated strings
Suspicious indicators	Manual interpretation	Automated indicators

How to generate the report: Show 3-5 matching fields to prove the same file is being compared, then explain the different strengths of the two tools.

Result / Conclusion: PEview and PESTudio were compared; PEview provides detailed PE structure, while PESTudio adds malware-focused interpretation and indicators.

Q2. Explain file property and header analysis. [15M]

Answer

File property and header analysis is a static-analysis method used to understand a suspicious file before execution. It examines general file metadata and the structured header fields that describe how the operating system should load the file.

- File properties include file name, size, type, extension, hashes, timestamps, digital signature, version information, and architecture.

- Hashes help uniquely identify the sample and compare it with known threat intelligence.
- File type checking confirms whether the real format matches the extension; a mismatch can be suspicious.
- DOS header analysis checks the MZ signature and e_lfanew pointer.
- PE signature confirms a valid Portable Executable structure.
- COFF/File Header provides machine type, number of sections, and timestamp information.
- Optional Header provides entry point, image base, subsystem, image size, section alignment, and data directories.
- Section table analysis reveals section names, virtual/raw sizes, addresses, and permissions.
- Unusual section names, writable+executable permissions, strange size differences, or high entropy can suggest packing or manipulation.
- Import/export tables can show external functions and capabilities.
- Resource and version information can provide icons, language, product names, or misleading metadata.
- Header values can be falsified, so conclusions should not rely on metadata alone.

Conclusion: Property and header analysis gives a fast structural picture of the file and helps decide whether deeper static or dynamic analysis is needed.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 19

Q1. Analyze a malware sample using VirusTotal and list the available IOCs. [15M]

Answer

Aim: To use a VirusTotal report to identify file, network, and behavior-related indicators associated with a malware sample.

Tools Required: VirusTotal and the given sample/hash.

Safe Setup: Follow lab policy before uploading unknown samples.

Step-by-Step Procedure

1. Open VirusTotal and search the sample SHA-256 or upload the file if permitted.
2. Record the detection ratio and primary file hashes.
3. Open Details and note file names, type, size, signatures, and other identifying metadata.
4. Open Relations/Behavior sections if available.
5. Collect file IOCs such as hashes, dropped-file hashes/names, and related executable names.
6. Collect network IOCs such as domains, URLs, IP addresses, and contacted hosts shown by the report.
7. Collect host IOCs such as registry keys, mutexes, process names, or file paths when shown.
8. Classify each IOC and include context so common legitimate values are not misrepresented.
9. Take screenshots of the relevant tabs.
10. Prepare an IOC table and a short verdict.

What to Observe / Record

- Hashes
- Names/paths
- Domains
- IPs
- URLs
- Dropped files
- Registry/process/mutex data if available

Report Format

IOC Type	Value	Source/Context
SHA-256	_____	VirusTotal Details
File name/path	_____	Relations/Behavior
Domain/URL	_____	Relations/Behavior
IP address	_____	Relations/Behavior
Other host IOC	_____	Behavior

How to generate the report: Not every VirusTotal report contains every IOC type. Write "Not available in report" when a category is absent instead of inventing a value.

Result / Conclusion: Available IOCs were extracted from VirusTotal and organized by type for investigation or threat hunting.

Q2. Explain malware classification basics. [15M]

Answer

Malware classification means grouping malicious software according to its behavior, purpose, spreading method, platform, or family. Correct classification helps analysts understand risk and choose suitable defenses.

- By behavior/purpose: ransomware, spyware, downloader, credential stealer, backdoor, bot, wiper, adware, etc.
- By spreading method: virus (attaches to files), worm (self-spreads), Trojan (disguised as legitimate software).
- By persistence/stealth: rootkits and bootkits focus on hiding or maintaining low-level access.
- By platform: Windows malware, Android malware, Linux malware, macOS malware, cross-platform malware.
- By delivery role: dropper installs a payload; downloader retrieves another payload; loader starts malicious code.
- By control model: bot malware connects infected systems to a botnet, often under remote command.
- By family: Samples sharing code, behavior, configuration, or infrastructure can be grouped into a malware family.

- Classification uses evidence from hashes, strings, imports, PE structure, signatures, behavior, network activity, and threat intelligence.
- One sample may fit more than one class. For example, malware may be both a Trojan and a credential stealer.
- Antivirus vendors may use different naming conventions, so family names can differ even for the same sample.

Conclusion: Malware classification is not only about giving a name; it summarizes how the malware works and what risk it creates.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 23

Q1. Prepare a malware analysis report based on the provided VirusTotal and PESTudio results. [15M]

Answer

Aim: To combine VirusTotal reputation/detection information and PESTudio static-analysis findings into one clear malware analysis report.

Tools Required: Provided VirusTotal result, provided PESTudio result, report document.

Safe Setup: Use only the evidence given in the practical. If a field is missing, state "Not available".

Step-by-Step Procedure

1. Create a report title containing the sample name/hash and analysis date.
2. Write an Executive Summary of 3-5 lines stating whether the sample appears malicious/suspicious and the main reasons.
3. Add Sample Information: file name, size, file type, architecture, MD5, SHA-256.
4. Add VirusTotal Findings: detection ratio, major vendor verdicts, reputation/relations, and any available IOCs.
5. Add PESTudio Findings: imports, suspicious APIs, indicators, strings, sections, compiler/packer information if shown.
6. Add an IOC section grouping hashes, file paths, registry keys, domains, URLs, IPs, and other indicators that are actually present.
7. Add Screenshots/Evidence references for important VirusTotal and PESTudio pages.
8. Write an Analysis/Interpretation section explaining how the evidence supports the verdict.
9. Add a Conclusion with the final assessment and what additional analysis would be useful if evidence is incomplete.
10. Proofread the report so every claim is supported by the supplied results.

What to Observe / Record

- VirusTotal detection
- File identifiers
- PESTudio indicators
- Imports/APIs
- Strings
- Sections/packer signs
- IOCs

Report Format

Report Section	What to write
1. Sample Information	Name, size, type, MD5, SHA-256, architecture
2. VirusTotal	Detection ratio, vendor results, known relations/IOCs
3. PESTudio	Imports, APIs, indicators, strings, sections
4. IOCs	Hashes, domains, IPs, URLs, paths, registry, etc.
5. Evidence	Screenshots / references
6. Conclusion	Malicious / Suspicious / Inconclusive + reasons

How to generate the report: Use facts first and interpretation second. Example: write "PEStudio shows URL X and WinINet imports" before writing "This may indicate network communication." Never invent missing sample values.

Result / Conclusion: A structured malware analysis report was prepared by combining the supplied VirusTotal and PESTudio evidence.

Q2. Explain the format of a malware analysis report. [15M]

Answer

A malware analysis report should be clear, repeatable, evidence-based, and easy for another analyst or examiner to follow. A good report separates observed facts from interpretation.

- Title and Case Information: Report title, date, analyst/student name if required, sample identifier.
- Executive Summary: Short overview of the sample, key findings, and final verdict.
- Sample Information: File name, size, type, architecture, hashes, compile metadata, and source if known.

- Analysis Environment: Tools used and, for dynamic analysis, VM/sandbox details.
- Static Analysis Findings: Hashes, PE headers, imports, sections, strings, packer/compiler, signatures, and suspicious indicators.
- Dynamic/Behavior Findings: Processes, files, registry changes, persistence, services/tasks, and memory behavior if performed.
- Network Findings: DNS, IPs, URLs, ports, protocols, and command-and-control behavior if observed.
- Indicators of Compromise: A clean table of hashes, domains, IPs, URLs, file paths, registry keys, mutexes, etc.
- Evidence: Screenshots, log references, exported reports, or PCAP files.
- Assessment/Interpretation: Explain what the findings mean and how strongly they support the verdict.
- Conclusion: State malicious/suspicious/inconclusive and summarize the strongest evidence.
- Recommendations: In a real incident, this may include isolation, blocking IOCs, scanning related hosts, patching, password resets, or recovery actions.

Conclusion: The best report is factual, organized, reproducible, and avoids unsupported claims.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 26

Q1. Analyze the suspicious executable using PESTudio and prepare a summary of suspicious findings. [15M]

Answer

Aim: To perform PESTudio static analysis and summarize the most important suspicious characteristics of the executable.

Tools Required: PESTudio and the given executable.

Safe Setup: Use only the sample provided by the examiner. For execution-based analysis, work inside an isolated VM/sandbox and keep personal files and shared folders disconnected.

Step-by-Step Procedure

1. Open the sample in PESTudio and note basic file information and hashes.
2. Review Indicators and note the highest-priority warnings.
3. Review Libraries and Imports for APIs related to networking, process/memory manipulation, registry changes, services, or other sensitive actions.
4. Review Strings for URLs, domains, IPs, paths, commands, registry references, suspicious extensions, or messages.
5. Review Sections for unusual names, high entropy, or suspicious permissions.
6. Check compiler/packer information and signatures if PESTudio provides them.
7. Check digital signature/version information and note whether it appears valid, absent, or suspicious.
8. Group findings by category instead of copying every PESTudio entry.
9. Take screenshots of the strongest findings.
10. Write a short final summary explaining why the file is suspicious and what additional dynamic analysis should verify.

What to Observe / Record

- High-priority PESTudio indicators
- Suspicious imports/APIs
- Network/command strings
- Section/entropy anomalies
- Packer/compiler info
- Signature/version info

Report Format

Category	Suspicious finding	Reason
Indicators	_____	_____
Imports/APIs	_____	_____
Strings	_____	_____
Sections/packing	_____	_____
Signature/metadata	_____	_____

How to generate the report: Select the strongest 5-10 findings, not every warning. A strong summary explains the evidence and avoids declaring malware from one indicator alone.

Result / Conclusion: PESTudio was used to identify and summarize the most important suspicious static findings in the executable.

Q2. Explain code obfuscation and malware packing. [15M]

Answer

Code obfuscation and packing are techniques used to make software harder to inspect. Malware commonly uses them to hide strings, logic, and signatures from analysts or security tools.

- Code obfuscation changes the representation of code without changing its intended behavior.
- Simple obfuscation includes encoded/encrypted strings, renamed symbols, junk instructions, or indirect API calls.
- More advanced obfuscation may alter control flow, resolve APIs at runtime, or construct code/data only during execution.

- Malware packing wraps the original executable in a compressed/encrypted form together with a small unpacking stub.
- When packed malware runs, the stub restores the original code in memory and transfers execution to it.
- Packing signs include high entropy, unusual section names/sizes, a small import table, and known packer signatures.
- Obfuscation signs include unreadable strings, unusual control flow, encoded configuration, and dynamic API resolution.
- Legitimate software can use obfuscation or packing for protection, so the technique alone does not prove malware.
- Analysts respond by identifying the packer, examining memory after unpacking, decoding strings/configuration, and using dynamic analysis.
- Static and dynamic analysis together are usually required because these techniques are specifically designed to hide static evidence.

Conclusion: Packing hides the original executable as a wrapped/compressed form, while obfuscation makes code and data difficult to understand. Both increase analysis difficulty.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.

Sheet 28

Q1. Capture malware network traffic using Wireshark and identify the communicating IP addresses and protocols. [15M]

Answer

Aim: To capture suspicious network communication and identify remote IP addresses, ports, and protocols used by the sample.

Tools Required: Wireshark, isolated/lab-controlled analysis network, given sample.

Safe Setup: Use a controlled lab network only. Do not expose an unknown sample directly to a normal personal or production network.

Step-by-Step Procedure

1. Open Wireshark and select the active interface used by the analysis VM.
2. Start capture before sample execution and note the start time.
3. Execute the sample in the isolated environment.
4. Allow enough time for network activity as instructed, then stop the capture.
5. Open Statistics > Endpoints to list IP addresses and Statistics > Conversations to see source-destination pairs.
6. Apply filters such as ip, tcp, udp, dns, http, tls, or ip.addr == x.x.x.x to inspect suspicious traffic.
7. Record destination IP addresses, destination ports, and identified protocols.
8. Check DNS packets to map domains to IP addresses where possible.
9. Distinguish sample-related traffic from normal background traffic using timestamps and the lab baseline.
10. Save the PCAP and include screenshots of important conversations/packets in the report.

What to Observe / Record

- Source IP
- Destination IP
- Source/destination port
- Protocol
- DNS/domain if present
- Connection frequency/timing

Report Format

Source IP	Destination IP	Port	Protocol	Observation
_____	_____	_____	TCP/UDP/DNS/ HTTP/TLS	_____
_____	_____	_____	_____	_____

How to generate the report: Use the Endpoints/Conversations views for the summary table, then show packet-level evidence for the most important connection. State uncertain traffic as "possibly related" rather than definite.

Result / Conclusion: Wireshark captured the network traffic and the communicating IP addresses, ports, and protocols were documented.

Q2. Explain the importance of network analysis during malware investigation. [15M]

Answer

Network analysis is a major part of malware investigation because many malware samples communicate with external infrastructure to receive commands, download payloads, spread, or send stolen information.

- Reveals command-and-control servers through recurring outbound connections, domains, or IP addresses.
- Identifies malicious or suspicious DNS requests.
- Shows destination IP addresses, ports, and protocols used by the malware.
- Can reveal additional payload downloads, redirects, or update mechanisms.
- Helps detect possible data exfiltration or large/unusual outbound transfers.
- Provides network IOCs that can be searched across firewall, proxy, DNS, IDS/IPS, and endpoint logs.
- Allows defenders to block malicious domains, URLs, IPs, or traffic patterns when appropriate.
- Helps correlate events: a process starts at a certain time and a new network connection appears immediately afterward.
- Can show propagation attempts such as SMB or other lateral network traffic.

- Helps scope an incident by identifying other hosts that contacted the same infrastructure.
- Limitations include encrypted traffic, fast-changing infrastructure, shared cloud/CDN IPs, and unrelated background traffic.

Conclusion: Network analysis turns malware communication into actionable evidence for detection, containment, threat hunting, and incident response.

Q3. Viva [5M]

No fixed written answer. Be ready to explain the tool used in Q1, the main findings, and the key terms from Q2 in 2-3 sentences.